Zuse 2 – Der Rechner steuert sich selbst

Konzept des zweiten Bandes der Zuse-Reihe: gespeicherter Ablauf, Speicher mit Adressen, Bus, Steuerwerk, Zuse-4 und Zuse-8 mit Assembler für 12 bis 16 Jahre

Dokument	Konzept
Version	0.1
Status	ENTWURF
Autor	Daniel Zeimentz · DC Software Engineering
Datum	14.11.2026
Bezug	Konzept Zuse v0.3 (Reihe, Abschnitte 4.1, 5.5, 5.6, 7.2); Konzept Zuse 3 v0.1; Konzept MCU Explorer v0.2; DC Learning Ecosystem v1.1, Design Standard v1.0; ADR-0005, ADR-0006, ADR-0010, ADR-0011
Zielplattformen	Windows zuerst, dann Android-Tablet, iPadOS, macOS, Linux (wie Zuse 1)
Entscheidungen	14.11.2026 (Inhaber): Steuerwerk erst fest verdrahtet (Zuse-4), dann als Mikroprogramm (Zuse-8); Zuse-8 mit eigenem Befehlssatz als Akku-Maschine

Inhalt

1. Zusammenfassung **3**

2. Ausgangslage **3**

2.1 Was Zuse 1 mitgibt . . . 3

2.2 Lehrplanbezug . . . 3

2.3 Anregungen (nur Prinzipien) . . . 3

3. Ziele und Nicht-Ziele **4**

3.1 Ziele . . . 4

3.2 Nicht-Ziele . . . 4

4. Lösungskonzept **4**

4.1 Leitidee: vom Lochstreifen zum gespeicherten Programm . . . 4

4.2 Der Zuse-4 . . . 5

4.3 Der Zuse-8 . . . 6

4.4 Steuerwerk: erst fest, dann Tabelle . . . 6

4.5 Programmierebenen . . . 7

4.6 Bedienung und Gestaltung . . . 7

5. Lernpfad **7**

6. Technische Architektur **9**

7. Datenschutz, Kindersicherheit, Barrierefreiheit **9**

8. Anforderungen **10**

9. Risiken und Maßnahmen **10**

10. Umsetzungsplan **11**

11. Offene Entscheidungen **11**

12. Fazit und Empfehlung **11**

1. Zusammenfassung

Zuse 1 endet mit einer Rechenmaschine aus eigenen Erfindungen, die das Kind Schritt für Schritt von Hand steuert (»Du bist das Steuerwerk«), und mit der Frage: **Wer gibt die Schritte, wenn niemand am Bedienfeld steht?** Zuse 2 beantwortet sie mit Konrad Zuses Idee des **gespeicherten Ablaufs**. Die Jugendlichen bauen einen Speicher mit Adressen, verbinden alles über einen Bus, bauen das Steuerwerk, das Befehle holt und ausführt, und bringen so den **Zuse-4** zum Laufen – einen 4-Bit-Rechner, der über das Bedienfeld programmiert wird.

In der zweiten Hälfte wächst der Rechner auf 8 Bit. Das Steuerwerk wird zur **Tabelle (Mikroprogramm)**, in die eigene Befehle eingetragen werden können. Es entsteht der **Zuse-8** mit eigenem Befehlssatz, Stapel, Unterprogrammen, Tasten und einer Pixelmatrix. Am Ende steht der Schritt vom Kippschalter zum **Assembler** und ein selbst programmiertes kleines Spiel.

Das Leitbild bleibt das Bedienfeld der ersten Bausatz-Rechner: Programme werden zuerst mit Schaltern eingegeben und an LEDs abgelesen, erst danach als Text geschrieben. Jeder Schritt bleibt sichtbar bis zum Gatter (»Blick hinein«).

Umfang: 9 Lernstufen mit 41 Experimenten, geschätzt 140–185 Personentage. **Empfehlung:** Start nach Zuse 1 Version 1.0, vorher ein kleiner Machbarkeitstest (Zustand Z in `logic/`, Zuse-8 als Verhaltensmodell, Mikroprogramm).

2. Ausgangslage

2.1 Was Zuse 1 mitgibt

Aus Zuse 1	Wiederverwendung in Zuse 2
Gatter, Rätselkisten, Werkbank, Werkzeugkiste, Blick hinein	unverändert; neue Rätselkisten (Bustreiber, Decoder)
Register mit Takt und Löschen, Zähler, Vergleicher, Rechenwerk (Plus und Minus)	Bausteine des Zuse-4 (Akku, Befehlszähler, Rechenwerk)
Experiment 5.3 »Du bist das Steuerwerk« mit Signaltabelle	dieselbe Tabelle wird in Stufe 3 von Gattern erzeugt und in Stufe 6 zum Mikroprogramm
Zahlenleiste (binär, dezimal, hex), Taktgeber, Forscherheft, Profile, Aufgabe der Lehrkraft, Vorführung	unverändert übernommen
Historische Entwicklung (Zeitleiste)	fortgesetzt: Lochstreifen der Z3, Speicherprogramm, Bausatz-Rechner der 1970er

Die Erfindungen aus Zuse 1 kommen auf zwei Wegen nach Zuse 2: Liegt eine Exportdatei aus Zuse 1 vor, übernimmt Zuse 2 die eigenen Erfindungen; sonst stehen die Musterbausteine bereit. Stufe 0 wiederholt das Nötige in kurzer Form, damit Zuse 2 auch ohne Zuse 1 begonnen werden kann.

2.2 Lehrplanbezug

Informatik Sek. I (ab Klasse 7/8) und Übergang Sek. II: Von-Neumann-Rechner, Speicher und Adressen, Befehlszyklus (Holen, Dekodieren, Ausführen), Maschinensprache und Assembler, Zahlendarstellung mit Vorzeichen (Zweierkomplement), Algorithmen mit Verzweigung und Schleife auf Maschinenebene. Die Zuordnung zu Ländern, Luxemburg und Ostbelgien folgt im Schulpaket.

2.3 Anregungen (nur Prinzipien)

- **Bedienfeld früher Bausatz-Rechner** (1975): Untersuchen, Ablegen, Einzelschritt.

- **Lehrsimulatoren einer Von-Neumann-Maschine** (z. B. Johnny-Simulator): Steuerwerk als aufklappbares Mikroprogramm, Holphase sichtbar, eigene Befehle anlegen. Übernommen wird nur das Prinzip; Befehlsnamen, Kodierung und Aufgaben sind eigen.
- **Lehrbuch-Rechner mit 16 Speicherplätzen** (8-Bit-Wort aus Befehl und Adresse): Vorbild für die Wortaufteilung des Zuse-4.
- **ELRAD-Lehrgang Digitaltechnik** (Auswertung in `docs/zuse1-planung.md`): Reihenfolge Decoder → Speicher → Bus → Steuerwerk.

3. Ziele und Nicht-Ziele

3.1 Ziele

- Verstehen, wie ein Rechner sich selbst steuert: Befehl holen, entschlüsseln, ausführen – an einem selbst gebauten Rechner.
- Jede Stufe beginnt mit einer Frage aus dem Alltag oder der Geschichte; jedes Experiment bringt nur einen neuen Gedanken.
- Programmieren zuerst am Bedienfeld (Binär, Bit für Bit), danach in Assembler-Text.
- Ehrliche Simulation: Der Zuse-4 läuft auf Gatterebene; der Zuse-8 läuft als Verhaltensmodell, das erst entsteht, wenn dieselbe Schaltung die Prüfung bestanden hat.
- Gleiche Gestaltung und Bedienung wie Zuse 1 und Volta (Design Standard, drei Schichten).

3.2 Nicht-Ziele

- Kein Nachbau eines realen Prozessors, Befehlssatzes oder Bausatz-Rechners.
- Keine Hochsprache (kommt als Einblick in Zuse 3), kein C/C++ (MCU Explorer).
- Keine Unterbrechungen, Zeitgeber-Bausteine, Ports als Speicheradressen, kein Cache (Zuse 3).
- Keine Elektrik (Spannungen, Ströme): Zuse bleibt rein digital.

4. Lösungskonzept

4.1 Leitidee: vom Lochstreifen zum gespeicherten Programm

Die Geschichte von Zuse 2 folgt der Technikgeschichte. Zuerst steuert ein **Lochstreifen** die Rechenmaschine aus Zuse 1 – wie bei der Z3, die ihre Befehle von einem gelochten Filmstreifen las und noch keine bedingten Sprünge kannte. Der Streifen läuft nur vorwärts. Dann wandern die Befehle in denselben **Speicher** wie die Zahlen, ein **Befehlszähler** zeigt auf den nächsten Befehl, und Sprünge werden möglich. Damit ist aus der Rechenmaschine ein Rechner geworden.

4.2 Der Zuse-4

Merkmal	Festlegung
Rechenwerk	4 Bit, Plus und Minus aus Zuse 1, Merker Übertrag (C) und Null (Z)
Register	Akku A (4 Bit), Befehlszähler (4 Bit), Befehlsregister (8 Bit), Adressregister (4 Bit), Ausgaberegister (4 Bit)
Speicher	16 Plätze zu 8 Bit; jedes Wort hat einen **Befehlsteil** (linke 4 Bit) und einen **Zahlteil** (rechte 4 Bit: Adresse oder Zahl)
Bus	ein gemeinsamer Bus mit Bustreibern (Zustand Z), kein eigener Adressbus
Steuerwerk	fest verdrahtet: Phasenzähler (4 Takte je Befehl), Befehlsdecoder, Steuermatrix aus Gattern
Bedienfeld	4 Adressschalter, 8 Datenschalter (Befehl · Zahl); LEDs für Adresse, Daten, A, C, Z, Ausgabe, Lauf/Halt; Taster Untersuchen, Ablegen, Ablegen-weiter, Einzelschritt (Takt oder Befehl), Start/Stopp, Rücksetzen
Simulation	flach auf Gatterebene (ca. 600 Gatter), Anzeigetakt bis 50 Hz, Einzelschritt je Takt

Die Aufteilung des Worts in Befehls- und Zahlteil klärt die offene Frage aus dem Arbeitsentwurf in Konzept Zuse 5.6 (16 Wörter zu 4 Bit reichten nicht für Befehl und Adresse). Der Datenweg bleibt 4 Bit breit; in Datenplätzen ist der Befehlsteil 0000.

Befehlssatz Zuse-4 (Kurzzeichen allgemein üblich, Klartext in jeder Sprache):

Code	Kurzzeichen	Klartext (DE)	Wirkung
0	HLT	Halt	Rechner hält an (leerer Speicher hält sofort)
1	LDA a	Lade	A ← Speicher[a]
2	ADD a	Addiere	A ← A + Speicher[a]; setzt C und Z
3	SUB a	Ziehe ab	A ← A − Speicher[a]; setzt C und Z
4	STA a	Speichere	Speicher[a] ← A
5	OUT	Zeige	Ausgabe ← A
6	JMP a	Springe	nächster Befehl bei a
7	JZ a	Springe, wenn null	wenn Z = 1: nächster Befehl bei a
8	JC a	Springe bei Übertrag	wenn C = 1: nächster Befehl bei a
9–F	–	frei	im Zuse-4 Halt; Raum für Experimente

Befehlszyklus in vier Takten: T0 Adresse aus dem Befehlszähler, T1 Befehl holen und Befehlszähler weiterzählen, T2 und T3 ausführen. Die LEDs »Holen« und »Ausführen« zeigen die Phase in Wort und Symbol.

4.3 Der Zuse-8

Merkmal	Festlegung
Rechenwerk	8 Bit aus zwei 4-Bit-Rechenwerken mit Übertrag; Plus, Minus, UND, ODER, XODER, Schieben; Merker Z, C, N (negativ)
Register	Akku A, Indexregister X, Befehlszähler (8 Bit), Stapelzeiger SP, Merker
Speicher	256 Plätze zu 8 Bit; **eigener Adressbus** (8 Bit) neben dem Datenbus
Befehle	1 oder 2 Byte (Befehl, ggf. Zahl oder Adresse); Adressarten unmittelbar (#n), direkt (a), indiziert (a,X)
Steuerwerk	**Mikroprogramm**: Tabelle aus Steuerwörtern, adressiert durch Befehl und Phase; Holteil für alle Befehle gleich
Ein-/Ausgabe	eigener E/A-Raum mit Befehlen IN/OUT (Ports als Speicheradressen erst in Zuse 3); Tasten, Schalter, Siebensegment, Pixelmatrix 8 × 8
Bedienfeld	8 Adress-, 8 Datenschalter; LEDs Adresse, Daten, A, X, Z, C, N, Status (Holen, Lesen, Schreiben, E/A, Stapel), Lauf/Halt; Taster wie Zuse-4 und Untersuchen-weiter
Simulation	Bausteine als Verhaltensmodelle nach bestandener Prüfung; Blick hinein bis zum Gatter bleibt; Programmtakt einstellbar 1 Hz bis 10 kHz

Befehlssatz Zuse-8 (Arbeitsentwurf, eigene Kodierung; Akku-Maschine):

Gruppe	Befehle
Transport	LDA #n / a / a,X · STA a / a,X · LDX #n / a · STX a · INX · DEX
Rechnen und Logik	ADD · SUB · AND · OR · XOR (jeweils #n / a / a,X) · SHL · SHR · CMP
Sprünge	JMP · JZ · JNZ · JC · JN
Stapel und Unterprogramme	PUSH · POP · CALL · RET
Ein-/Ausgabe	IN p · OUT p
Steuerung	NOP · HLT

Insgesamt 24 Grundbefehle. Eigene Befehle entstehen im Mikroprogramm (Stufe 6) auf freien Codes und erscheinen danach im Assembler mit eigenem Kurzzeichen.

Pixelmatrix: 8 × 8 Leuchtpunkte mit eigenem Bildspeicher; Port 1 wählt die Zeile, Port 2 schreibt das Muster (ein Byte = eine Zeile). Tasten liegen an Port 0 (vier Richtungstasten, eine Aktionstaste), die Siebensegmentanzeige an Port 3. Leuchtpunkte zeigen ihren Zustand zusätzlich als Form (gefüllt/leer), nicht nur über Farbe.

4.4 Steuerwerk: erst fest, dann Tabelle

1. **Fest verdrahtet (Stufe 3, Zuse-4):** Der Phasenzähler (Zähler aus Zuse 1 mit Decoder) liefert T0 bis T3, der Befehlsdecoder »1 aus 16« den Befehl. Eine Steuermatrix aus UND/ODER verknüpft beides zu den Steuersignalen (z. B. »Speicher auf den Bus«, »A übernimmt«). Die Matrix ist teilweise vorgegeben. Das Kind ergänzt die fehlenden Verbindungen und prüft sie gegen die Signaltabelle aus Zuse-1-Experiment 5.3. Damit erzeugen Gatter dieselben Signale, die das Kind dort von Hand gegeben hat.
2. **Mikroprogramm (Stufe 6, Zuse-8):** Die Signaltabelle wird selbst zum Speicher. Jede Zeile ist ein **Steuerwort** (ein Bit je Steuersignal), adressiert durch Befehl und Phase. Die Tabelle erscheint als Raster mit Häkchen und Signalnamen. Ein eigener Befehl entsteht durch Ausfüllen freier Zeilen, etwa »Verdopple« (A + A), »Tausche A und X« oder »Lade und zähle weiter«. Die Prüfung führt den neuen Befehl auf Testwerten aus und vergleicht mit der beschriebenen Wirkung.

Gründe für die Reihenfolge: Das feste Steuerwerk zeigt, dass auch die Steuerung nur aus Gattern besteht. Die Tabelle zeigt, warum größere Rechner anders gebaut werden: Eine Zeile ist leichter zu ändern als eine Verdrahtung.

4.5 Programmierebenen

Ebene	Ab	Form
Bedienfeld	Stufe 4	Adresse einstellen, Befehl und Zahl an den Schaltern, Ablegen; Programmtabelle zeigt Adresse, Binärcode, Kurzzeichen, Klartext, Wirkung
Befehlsleiste	Stufe 4	Befehle aus einer Leiste antippen statt Bits einzustellen (Touch); Schalter folgen sichtbar
Assembler	Stufe 8	Text mit Marken, Kommentaren, Zahlen in binär/dezimal/hex; Fehlermeldungen in Kindersprache mit Zeile; Einzelschritt markiert die Zeile; Haltepunkte
Datei	Stufe 8	Speicherabbild als Intel HEX exportieren und importieren (nur als Datei, ohne Netz)

```
; Zuse-8: Punkt mit den Tasten über die Pixelmatrix bewegen (Arbeitsentwurf)
start:  LDA #8        ; Spalte 4 (Bitmuster 00001000)
        STA punkt
loop:   IN 0          ; Tasten lesen
        AND #1        ; rechts gedrückt?
        JZ zeige
        LDA punkt
        SHR           ; Punkt eine Spalte nach rechts
        STA punkt
zeige:  LDA #3
        OUT 1         ; Zeile 3 wählen
        LDA punkt
        OUT 2         ; Muster schreiben
        JMP loop
punkt:  0
```

4.6 Bedienung und Gestaltung

- Drei Schichten wie Zuse 1 (Konzept Zuse 6.3): Eingänge (Bedienfeld 4/8 Bit, Tasten), Logik (Register, Bus, Speicher, Steuerwerk), Ausgänge (LED-Zeilen, Siebensegment, Pixelmatrix).
- **Busse** als breite Bänder mit Breitenangabe (D[7:0]); Splitter und Bündel als eigene Werkzeuge. Ein Bus, auf dem zwei Treiber gleichzeitig senden, zeigt »Konflikt« in Wort und Zeichen (X).
- **Speicheransicht** neben dem Bedienfeld: 16 bzw. 256 Plätze als Tabelle (Adresse, Binär, Hex, Kurzzeichen); der aktuelle Befehl ist markiert (Rahmen und Pfeil, nicht nur Farbe).
- **Zeitdiagramm** (Logikanalysator aus Zuse 1, Phase Z2) für Takt, Phasen und Steuersignale.
- Einzelschritt je Takt oder je Befehl; »Lauf« mit Taktwahl (1 Hz, 10 Hz, 100 Hz, 1 kHz, 10 kHz beim Zuse-8).
- Zetti erklärt Holen und Ausführen beim ersten Mal mit einer kurzen Animation des Datenwegs (abschaltbar).

5. Lernpfad

Wie Zuse 1 in Lernstufen mit Leitfrage; Experimentarten **entdecken**, **bauen**, **zahlen** (fest verdrahtet bedienen) und neu **programmieren** (Programm schreiben, Prüfung über erwartete Ausgabe bzw. Speicherinhalt nach dem Halt, mit mehreren Testeingaben).

Stufe	Leitfrage	Experimente (Art, Ergebnis)
0 Auftakt	Wer gibt die Schritte?	0.1 Die Rechenmaschine von gestern (zahlen: Wiederholung »Du bist das Steuerwerk«) · 0.2 Der Lochstreifen (zahlen: Schritte auf einem Streifen steuern die Rechenmaschine; nur vorwärts)
1 Speicher mit Adressen	Welcher Platz ist gemeint?	1.1 Einer von vier (entdecken: Decoder »1 aus 4«) · 1.2 Vier Plätze (bauen: vier Register mit Decoder) · 1.3 Einer von sechzehn (bauen: Decoder aus zwei »1 aus 4«) · 1.4 Untersuchen und Ablegen (zahlen: Speicher 16 × 8 am Bedienfeld) · 1.5 Ablegen-weiter (bauen: Adresszähler)
2 Ein Bus für alle	Wie reden viele Bausteine über eine Leitung?	2.1 Zwei reden gleichzeitig (bauen: Buskonflikt, Pegel X) · 2.2 Der Torwächter (entdecken: Bustreiber mit Zustand Z) · 2.3 Der Datenweg (zahlen: Akku, Rechenwerk, Speicher am Bus, Freigaben von Hand)
3 Das Steuerwerk	Wie holt sich der Rechner seinen nächsten Befehl?	3.1 Der Streifen wird Speicher (bauen: Befehlszähler mit Laden) · 3.2 Befehl holen (bauen: Befehlsregister) · 3.3 Ein Befehl in vier Takten (bauen: Phasenzähler mit Decoder) · 3.4 Welcher Befehl ist es? (bauen: Befehlsdecoder) · 3.5 Wer gibt die Signale? (bauen: Steuermatrix, teilweise vorgegeben)
4 Läuft er?	Was kann der Zuse-4?	4.1 Das erste Programm (programmieren: 6 + 3 am Bedienfeld) · 4.2 Holen und Ausführen (zahlen: Einzelschritt je Takt beobachten) · 4.3 Ergebnisse ablegen (STA) · 4.4 Endlos zählen (JMP) · 4.5 Countdown (JZ) · 4.6 Größer oder kleiner? (JC nach SUB; der Vergleicher kehrt wieder) · 4.7 Malnehmen ohne Malnehmen (Projekt: wiederholte Addition)
5 Mehr Bits	Wie wird aus 4 Bit ein 8-Bit-Rechner?	5.1 Bündel und Splitter (bauen: 8-Bit-Busse) · 5.2 Doppelt rechnen (bauen: 8-Bit-Rechenwerk aus zwei 4-Bit-Rechenwerken) · 5.3 Zahlen unter null (zahlen: Zweierkomplement, Merker N) · 5.4 Ein eigener Weg für Adressen (bauen: Speicher 256 × 8 mit Adressbus)
6 Das Steuerwerk als Tabelle	Wie baut man ein Steuerwerk, das man ändern kann?	6.1 Die Tabelle wird Speicher (zahlen: Mikroprogramm statt Matrix, gleiche Signale) · 6.2 Holen ist immer gleich (bauen: gemeinsamer Holteil) · 6.3 Ein eigener Befehl (bauen: »Verdopple«) · 6.4 Befehle mit zwei Bytes (bauen: unmittelbare Zahl #n) · 6.5 Der Zuse-8 (zahlen: vollständiger Rechner, Verhaltensmodell nach Prüfung)
7 Merken, Zurückkehren, Antworten	Wie arbeitet ein Programm mit Listen, Unterprogrammen und Tasten?	7.1 Der Zeiger (programmieren: Indexregister, Liste summieren) · 7.2 Der Stapel (PUSH/POP) · 7.3 Hin und zurück (CALL/RET) · 7.4 Tasten lesen (IN) · 7.5 Die Pixelmatrix (OUT) · 7.6 Zahlen anzeigen (Siebensegment)

Stufe	Leitfrage	Experimente (Art, Ergebnis)
8 Vom Schalter zum Assembler	Warum schreibt man Programme als Text?	8.1 Text statt Schalter (programmieren: Assembler mit Marken) · 8.2 Fehler finden (Haltepunkte, Einzelschritt je Zeile) · 8.3 Programm als Datei (Intel HEX) · 8.4 Mein Spiel (Projekt: eigenes kleines Spiel auf der Pixelmatrix nach Vorschlägen, z. B. Punkte fangen oder Lauftext)

Summe: 9 Lernstufen, 41 Experimente, dazu Freies Labor (Zuse-4 und Zuse-8 frei programmierbar, Werkbank mit allen Werkzeugen) und Vorführung.

Freischaltung wie Zuse 1. Die Projekte 4.7 und 8.4 haben mehrere richtige Lösungen; geprüft wird nur das Verhalten (Ausgabe für Testeingaben), nie der Programmtext.

6. Technische Architektur

Bereich	Umsetzung
`logic/`	Erweiterung um Zustand **Z** (hochohmig) und Bustreiber; Konflikt zweier Treiber ergibt X; Speicherbaustein als Verhaltensmodell (16 × 8, 256 × 8); Verhaltensmodelle aus geprüften Erfindungen (Prüfung bestanden → schnelles Modell, Blick hinein zeigt weiter die Gatter)
Rechnermodell	Zuse-4 als Netzliste aus Erfindungen; Zuse-8 als Registertransfer-Modell mit Mikroprogramm-Tabelle, das gegen dieselbe Tabelle auf Gatterebene geprüft wird
Assembler	eigener kleiner Zweipass-Assembler (Qt-frei, C++17) mit Marken und Fehlermeldungen je Sprache; Kurzzeichen und Klartext aus einer Befehlstabelle (JSON), damit eigene Befehle sofort mitkommen
App	gleiche Codebasis wie Zuse 1 mit Ausgabe `zuse2` (Muster: `VOLTA_EDITION`), eigenes Symbol, App-ID `com.dcsoftwareengineering.zuse2`; Inhalte in `zuse/content/zuse2`
Datenformate	Stufen wie Zuse 1 (`stufen.json`), dazu Programme (Speicherabbild, Assembler-Text), Mikroprogramm (Tabelle), Intel HEX
Befehlsmuster	Werkbank wie Zuse 1; Mikroprogramm-Änderungen und Speicher-Ablegen im freien Labor ebenfalls als Befehle (Rückgängig); Programmlauf, Einzelschritt, Schalter sind Bedienhandlungen
Tests	Befehlssatz-Tests je Befehl (Zuse-4 Gatterebene gegen Registertransfer-Modell), Assembler-Tests, Smoke-Test aller Musterlösungen

Leistung: Zuse-4 flach mit ca. 600 Gattern bei 50 Hz Anzeigetakt ist nach den Erfahrungen mit Zuse 1 unkritisch. Für den Zuse-8 mit 10 kHz Programmtakt auf dem Referenztablet ist das Verhaltensmodell nötig; das prüft der Machbarkeitstest.

7. Datenschutz, Kindersicherheit, Barrierefreiheit

Wie Zuse 1 und Volta: offline, kein Konto, keine Analyse; Programme und Erfindungen nur lokal bzw. als Datei. Bedienelemente ≥ 48 dp, Farbe nie alleiniger Informationsträger (Pegel als 1/0/X/Z, Phasen als Wort). Sprachen DE, FR, EN, LU; Kurzzeichen der Befehle bleiben gleich, Klartext und Fehlermeldungen werden übersetzt.

8. Anforderungen

ID	Anforderung	Priorität
A1	Zustand Z und Bustreiber in `logic/`, Konflikte sichtbar	MUSS
A2	Zuse-4 auf Gatterebene mit Bedienfeld, Speicheransicht, Programmtabelle, Einzelschritt je Takt/Befehl	MUSS
A3	Mikroprogramm-Tabelle bearbeitbar, eigene Befehle mit Prüfung	MUSS
A4	Zuse-8 mit 24 Grundbefehlen, Stapel, IN/OUT, Pixelmatrix, Siebensegment, Tasten	MUSS
A5	Assembler mit Marken, Haltepunkten, verständlichen Fehlermeldungen	MUSS
A6	Intel HEX Export/Import	SOLL
A7	Übernahme eigener Erfindungen aus Zuse 1 (Datei)	SOLL
A8	Lernpfad 9 Stufen / 41 Experimente mit Hilfen und Musterlösungen	MUSS
A9	Forscherheft, Profile, Elternbereich, Aufgabe der Lehrkraft, Vorführung wie Zuse 1	MUSS
N1	Zuse-8 mit 10 kHz Programmtakt auf dem Referenztablet	MUSS
N2	Offline, keine Datenerhebung	MUSS

9. Risiken und Maßnahmen

Nr.	Risiko	Eintritt	Auswirkung	Maßnahme
R1	Steuerwerk (Stufe 3) für 12-Jährige zu schwer	MITTEL	HOCH	Matrix teilweise vorgegeben; Bezug zur bekannten Signaltabelle; gestufte Hilfen; Erprobung
R2	Sprung von Zuse-4 zu Zuse-8 zu groß	MITTEL	MITTEL	Stufe 5 baut nur die Breite aus; neue Befehle erst über das Mikroprogramm
R3	Verhaltensmodell weicht von der Gatterschaltung ab	NIEDRIG	HOCH	Modell entsteht nur aus bestandener Prüfung; automatische Vergleichstests
R4	Assembler am Tablet mühsam	MITTEL	MITTEL	Befehlsleiste zum Antippen, Marken aus Liste, Bildschirmtastatur mit Assembler-Zeichen
R5	Spiele-Projekt verleitet zum Nachbau bekannter Spiele	NIEDRIG	MITTEL	eigene Vorschläge; keine Namen oder Figuren bekannter Spiele in Inhalten
R6	Aufwand konkurriert mit Volta 3 und Zuse 1	HOCH	HOCH	Start erst nach Zuse 1 Version 1.0; Machbarkeitstest vorab

10. Umsetzungsplan

Phase	Inhalt	Grobschätzung
Z2-0	Machbarkeitstest: Zustand Z in `logic/`, Zuse-4 flach, Zuse-8 als Verhaltensmodell mit Mikroprogramm, Leistung Desktop und Tablet	10–15 PT
Z2-1	Ausgabe `zuse2`: Grundgerüst aus Zuse 1, Bedienfeld 8 Bit, Speicheransicht, Programmtabelle, Zeitdiagramm	20–25 PT
Z2-2	Stufen 0–4: Speicher, Bus, Steuerwerk, Zuse-4	35–45 PT
Z2-3	Stufen 5–6: 8 Bit, Mikroprogramm, Zuse-8	25–35 PT
Z2-4	Stufen 7–8: Stapel, Ein-/Ausgabe, Pixelmatrix, Assembler, Intel HEX	30–40 PT
Z2-5	Übersetzungen, Lehrerhandreichung, Erprobung, Paketierung	20–25 PT

Summe Zuse 2: ca. 140–185 PT. Wie bei Zuse 1 zuerst als Windows-Anwendung, danach Tablet.

11. Offene Entscheidungen

Entscheidung	Spätestens
Produktzusatz (Vorschlag: »Zuse 2 – Der Rechner steuert sich selbst«), Markenrecherche gemeinsam mit Zuse 1	vor Veröffentlichung
Altersangabe 12–16 oder 13–16 (nach Erprobung Zuse 1 und Verlagsrückmeldung)	vor Phase Z2-5
Kurzzeichen: international (LDA, ADD …) für alle Sprachen oder übersetzt	Phase Z2-1
Codierung des Zuse-8-Befehlssatzes und Anzahl freier Codes für eigene Befehle	Phase Z2-0
Gemeinsame Codebasis mit Ausgaben `zuse1`/`zuse2`/`zuse3` (Empfehlung) oder getrennte Projekte	Phase Z2-0

12. Fazit und Empfehlung

Zuse 2 schließt die Lücke zwischen der von Hand gesteuerten Rechenmaschine und dem programmierbaren Rechner, ohne einen Schritt zu überspringen. Lochstreifen, Speicher, Bus und Steuerwerk entstehen in der Reihenfolge der Geschichte und aus den Erfindungen von Zuse 1. Das Steuerwerk wird erst als Verdrahtung, dann als Tabelle gebaut und zeigt so zwei Bauweisen echter Prozessoren. Mit Assembler und Pixelmatrix endet der Band bei einem eigenen Spiel und mit der Frage, die Zuse 3 beantwortet: **Wie reagiert ein Rechner auf die Welt, ohne ständig nachzufragen?**

Nächste Schritte: (1) Konzept bestätigen. (2) Zuse 1 bis Version 1.0 fertigstellen. (3) Machbarkeitstest Z2-0.